Table of Contents

Abstract 1
1. Introduction 2
2. Background 4
3. Innovation and Significance 5
3.1 Project Innovations 5
3.2 Significance 5
4. Methodology 7
5. Experiment, Evaluation & Usability Study 9
5.1 Testing 9
5.2 Expectations 11
5.3 User Manual 12
6. Discussion 17
6.1 Ethics 17
6.2 Sustainability 18
6.2 Data Security 19
6.2 Project Management 20
7. Conclusion 21
Appendices 22

Abstract

The initial project idea was to develop a mobile application, called RosterFlow, using React Native as the front-end Framework and Java Spring Boot as the backend with a connection to a PostgreSQL database to store the information. The initial goal was to implement basic functionalities to roster users for various shifts. The application works on three levels: admin, company and user. The admin can create the company users, which are different companies that want to use this rostering service, and in turn, a company user can create users and assign them to shifts based on their availability. Users can view their rostered times and update their availability through the app. The use of agile development is implemented using one-week sprints, starting with a sprint planning meeting where everyone is assigned tasks, followed by an implementation stage, and finishing with a sprint review meeting where everyone highlights their progress. This process will be iterated until it is completed.

Version control is performed separately for the front-end and back-end, and hosted on BitBucket for everyone in the group to access. When someone needs to implement a feature, a branch will be made, and after everything has been reviewed and tested, the branch will be merged back into the main. This process was followed, but the project is still in progress as there were multiple delays, mainly due to members not contributing enough to the project goal.

A prototype was made with basic functionalities for shift allocations, and everything is working, but it needs more work to be deployment-ready, especially if we are talking about testing and edge-case scenarios. Basic unit testing for the backend APIs was implemented, along with integration testing. However, front-end testing and system-wide testing, which follow the procedure culminating in user shift allocation, still need to be implemented. Also, additional security measures have to be implemented, such as logging of access to the database and encryption for user data.

From experiments with the application, an issue was found with the reset password email. The hosting on Render does not allow sending emails, so we had to switch to an alternative service that is currently working, although the email is being directed to the spam folder. A usability study has not been done for time constraints, and also because the application is still in the development and testing stage, so doing it now would require us to redo it later, thus doing double the work.

1. Introduction

Nowadays, many companies still use email, texts, paper, word of mouth, and other tools that are not centralised to handle the rostering of employees. Our first goal is to replace any existing tools used, especially for small to medium-sized companies that generally do not use rostering applications for several reasons: cost, difficulty in adoption for administrative staff, and employees.

Our application goal is to be very easy to use and also provide functionalities that will simplify things for both company admins and employees (message board and unavailability), thereby reducing admin time and, consequently, costs. The project aims to create a Mobile application to handle rostering, used by multiple companies as a platform where they can create employees by simply entering an email address and assign them shifts according to their availability.

Ideally, the admin can manage everything in the application, but information related to company users would be accessible only to the company itself. The company is able to handle user creation, deletion, and also to revoke the credentials for a user to block him/her from accessing the app. Shift rostering is managed by the company's main account, but can also be performed by sub-company users, who are created by the company's main profile and must be approved.

The goal of this project is to create an easy-to-use and user-friendly application, both from the point of view of the company office staff, who have to use the rostering functionality of the app, plus other useful features like a messaging board, and the employees, who can easily look up their rostered times and put unavailability in the app. This, in turn, improves productivity, saves time, and ultimately saves the company money.

We aim to cover a basic process: company creation from the admin, now the company user can log in and utilise its functionalities. A company must set up a profile to create users, including both sub-company users and employees, to roster. When a user is created, it is assigned to only one company (for now). After this process is done and a user is created, the company can assign shifts to the user based on his/her availability.

This is a straightforward process that requires only two steps from the company to begin assigning shifts to a user. Once that is done, the shifts can also be deleted and modified to suit the company's needs or employees' unexpected changes in availability.

Additional functionalities would include a messaging board, which is currently partially implemented in the front-end and fully implemented in the back-end, but requires proper testing. Another functionality that needs to be implemented is the push notifications, but I would also argue that there should be both a push notification and an email when a shift is assigned and/or modified in order to account for malfunctions in either one of the two services to guarantee the user is notified

2. Background

There are already existing applications with similar functionalities to this project, such as Deputy. However, our focus is more directed to small to medium companies to replace existing methods that are in place for rostering with a digital counterpart. In this project we are using Java Springboot for the backend and React Native for frontend, specifically we started using the CLI but had to switch to Expo because there were some issue that were slowing down the implementation

There are numerous challenges associated with this project. First of all, it is not just a product for a single company; it is designed with the understanding that multiple companies can utilise it. This presents some challenges, but also motivates us, as it will be a valuable learning experience. Data privacy and security are other key issues, and PII must be encrypted to comply with Australian laws. Additionally, we need to log and restrict access to this information in case of a breach, to simplify the auditing process.

Usually, Shared IDs are not well accepted and should be avoided. However, let’s consider this application: if a company wants to create shifts for a user, it can log in and do just that. Now the question is: who logged in? Someone might say the company, but that is too broad, as it could be any of the company's employees, and this is very bad in the case of a breach.

To resolve this problem, a sub-company user was created, allowing every company employee to have a company user. This will make it easier to identify whose credentials were leaked in the event of a breach. Of course, now the problem is security, because if the credentials are actually leaked, we need to restrict the sub-company user's capabilities. This is done by allowing them to create a shift, which then has to be approved by the main company user (i.e., restricting access).

3. Innovation and Significance

3.1 **Project Innovations**

RosterFlow introduces several key innovations that set it apart from standard rostering apps. On the one hand, the app combines multiple features, including shift management, unavailability submissions, and a messaging board, all in one app. Users and managers would have to use separate tools for each task, which would make things confusing. With RosterFlow, everything is combined into a single app, which eliminates conflicts and fosters clearer communication among everyone involved within the organisation.

Another big feature is unavailability management. This means that when someone says they can't work a shift, the app updates it in real-time. So, managers can immediately identify if there is a clash or if a shift needs to be made by other users, which reduces mistakes and lowers stress within the organisation.

A unique feature of RosterFlow is its message board system. Staff can view posts and comment on them, while management can share announcements or communicate with everyone at once. There is also a filter that lets people show more or less content. This leads to better engagement and clearer workflows, allowing people to communicate effectively and concisely.

The app's layout focuses on user experience, with four main tabs (Home, Shift, Message, Profile). This makes it simple for users to navigate the app, and the login will direct them straight to their assigned location based on the provided roles. All the app pages are connected to the backend, which keeps everything updo date.

One additional point to note is that the app architecture was designed with future growth in mind. Using a backend-ready design means new features can be added without having to start from scratch. This is particularly helpful in a fast-moving work environment.

3.2 **Significance**

One of the benefits is reducing staff stress and burnout. In jobs where people are generally swapping shifts and finding replacements at the last minute, having a reliable way to submit unavailability and get fair shift allocation is crucial. RosterFlow supports this by providing staff with

an easy way to submit their absence requests, and managers receive instant updates. This will reduce the amount of manual checking and messaging back and forth, and help make sure nobody gets exhausted.

Another important aspect is organisational efficiency. Because the app connects everyone and updates in real-time-managers avoid things like accidentally placing people in double shifts. All these help organisations save time and reduce errors in scheduling.

The app is also designed to promote fairness. Shift allocations are transparent, and all users can see what is happening with their requests. This encourages fairer treatment overall and makes sure shifts are not allocated in a way that favours certain staff over others.

From a sustainability point, changing from paper rosters and unmanaged communications is also a win. This not only helps the environment but also reduces admin work and the risk of losing important documents.

Considering a broader impact, particularly in areas such as aged care, having better-managed staff and clearer communication leads to enhanced service delivery. Vulnerable groups, like elderly residents and patients, can benefit from having care staff who are well-scheduled and less stressed, which will improve the quality of care provided.

4. Methodology

Software Model and Development Process

The development of the RosterFlow system followed an Agile and Scrum approach. The work was broken down into weekly sprints, which were completed on a weekly basis as we reported on the corresponding weekly reports. Our project has two main parts. One part is the backend, and another part is the frontend, which each team member can work on.

During the project, we utilised Jira for Agile development and Bitbucket for version control, as well as Render.com for deployment. This allowed our team to work efficiently and effectively, identifying and resolving problems promptly.

Tech Stack

- **Programming Languages:** Java and JavaScript
- **Frameworks:** Spring Boot and React Native
- **Database:** H2 Database
- **Email Service:** MailJet API
- **Tools:** VSCode, IntelliJ IDEA, Maven, Postman, Jira, Bitbucket, Git Client, Render.com, Swagger

Architectural Design

The project was designed using a three-layer(**MVC**) architecture:

1. **Controller Layer**: handles client(Frontend) requests
2. **View Layer(Frontend)**: User Interface
3. **Model(Data Layer)**: manages data, connects to the database and handles queries

This structure helps keep the code clean, easy to understand, and easier to maintain in the future.

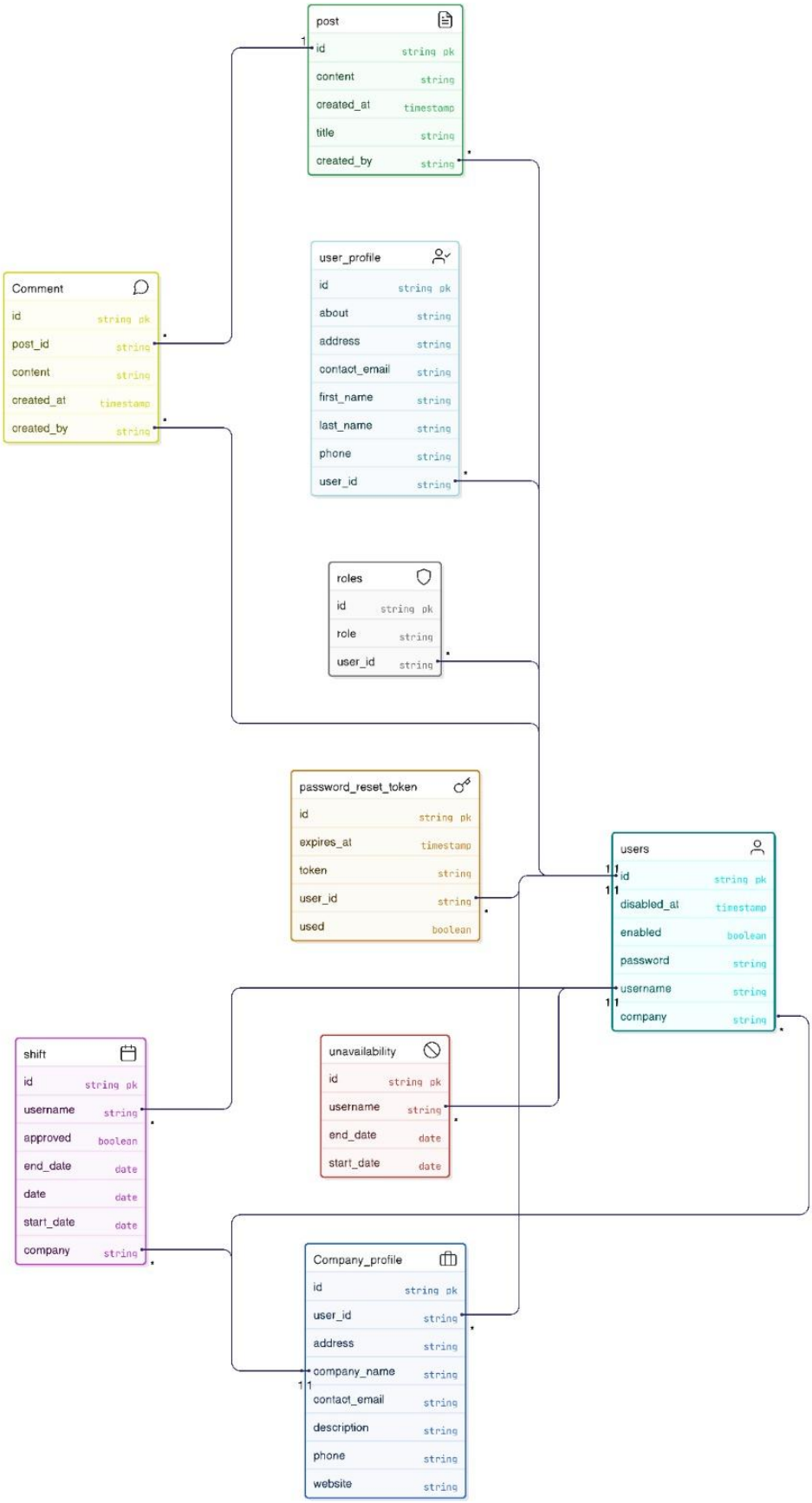


Figure 1. Database UML

5. Experiment, Evaluation & Usability Study

5.1 Testing

RoosterFlow had been developed in Java Spring Boot for the backend. While the front-end was developed using the React Native framework, it was built with Expo. Due to the nature of the software, exhaustive testing was not done. Random Testing was also skipped due to its inefficiency. Most of the tests were black-box testing, where the code structure was ignored and the test cases were based solely on the input and expected outputs.

The testing were done on all the controllers which were used in the backend. AuthController which deals with the authentication of user, company sign-up and sign-in. PasswordResetController which deals with sending a forgot password request, validation of the token and the reset of password if the token is validated. ShiftController deals with the company assigning shift for users assuming there is no unavailability on the time being assigned. UnavailabilityController is where a user can put the timeslot where they would be unavailable. During that time, shift can’t be assigned to them. UserShiftController displays the shifts that the particular user has.

Module(Unit) Testing

Module Testing was also done for the backend code. The modules was depending on other modules for particular information or task. To deal with this, the modules that were being depended on were stubbed to ensure smooth flowing of the test.

```java
@Autowired
MockMvc mockMvc;

@MockitoBean
ShiftRepository shiftRepo;

@MockitoBean
UserRepository userRepo;

@MockitoBean
private AuthenticationManager authenticationManager;

@MockitoBean
private PasswordEncoder encoder; //

@MockitoBean
private JwtUtil jwtUtils; //

@MockitoBean
private UnavailabilityRepository unavailabilityRepository;
```

Figure 2. Example of the modules being stubbed in module testing

The input that would be received from the frontend was validated throughout this test. In other controller tests, due to its complexity, mocking the modules was not possible; hence, module testing alone had to be abandoned.

Integration Testing

With a Top-Down Approach, the integration testing combines the ShiftController and UnavailabilityController.

```java
@Test
@WithMockUser(roles = "USER")
void addUnavailabilityTest() throws Exception {…


@Test
@WithMockUser(roles = "USER")
void modifyUnavailabilityTest() throws Exception {…


@Test
@WithMockUser(roles = "COMPANY")
void modifyUnavailability_overlapShift() throws Exception {…

@Test
@WithMockUser(roles = "COMPANY")
void modifyUnavailability_overlapUnavailability() throws Exception {
    Unavailability Unavailability = new Unavailability();
    Unavailability.setId(id:123L);
    Unavailability.setUsername(username:"user1@example.com");
    Unavailability.setStartDate(LocalDateTime.now().plusHours(hours:1));
    Unavailability.setEndDate(LocalDateTime.now().plusHours(hours:2));

    when(unavailabilityRepository.existsByUsernameAndStartDateLessThanEqualAndEndDateGreaterThanEqualAndIdNot(
        anyString(), any(), any(), any())).thenReturn(true);
```

Figure 3. ShiftController and UnavailabilityController is used together in Integration Testing

Manual Testing

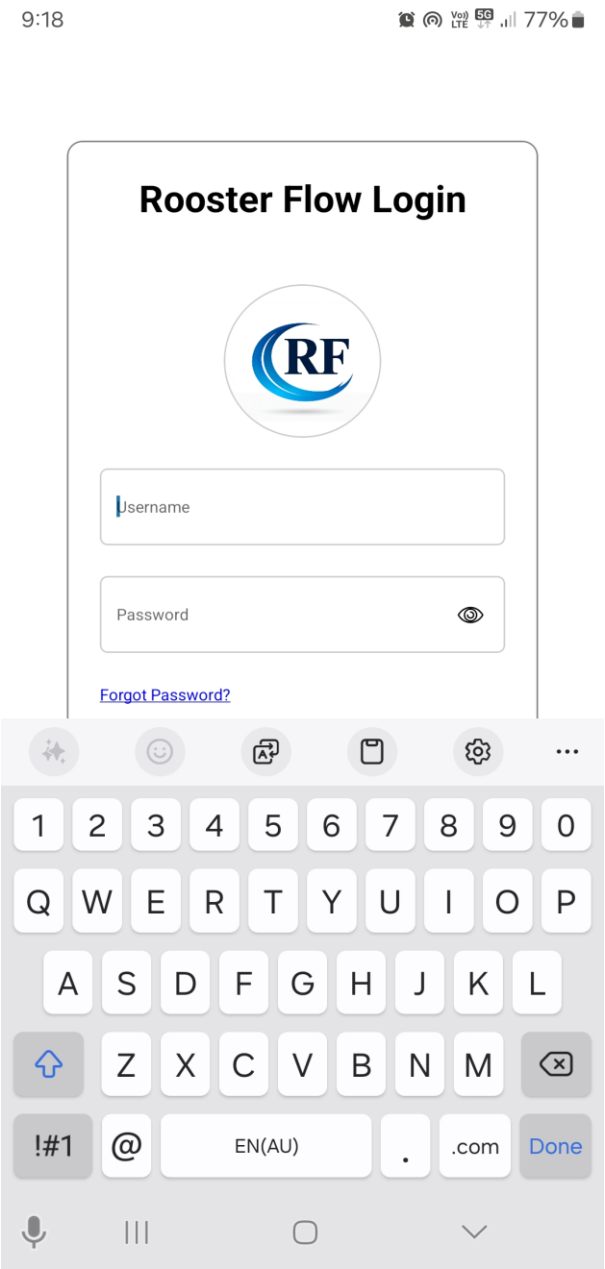
In the initial stages of the development of code, to check the functionality of the code, manual testing was done. Code at the initial stage were run and given sample inputs. Manual Testing was also done for bug fixes through out the code development.

5.2 Expectations

Despite trying our best, we were not able to reach the initial expectations we had set in place when we started the project. Language Inexperience was one of the major factor for the delay, with the unfamiliarity for some members, the tasks got delayed. The other issue was the time constraint as the features we had in mind took much longer than we expected.

5.3 User manual

The application has four users: admin, company, sub company and user. The admin user for now can only create company users, so every user is a prospective company. More methods will be implemented in future work as the admin should have control over the whole application. On the bottom login and company sign up screens.



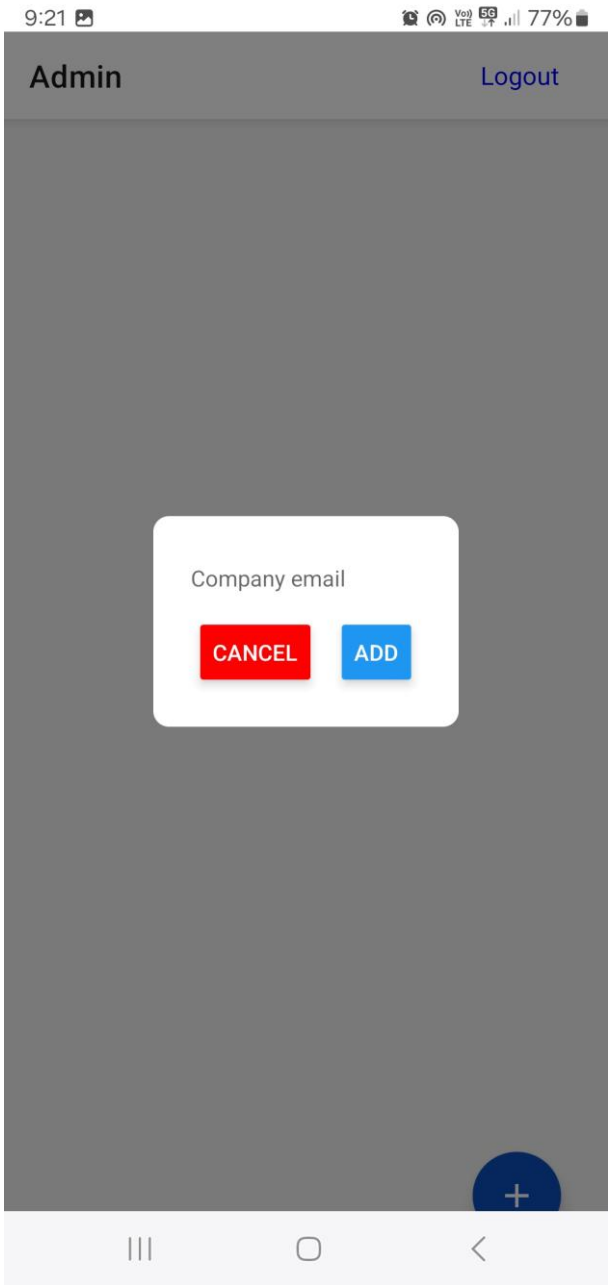
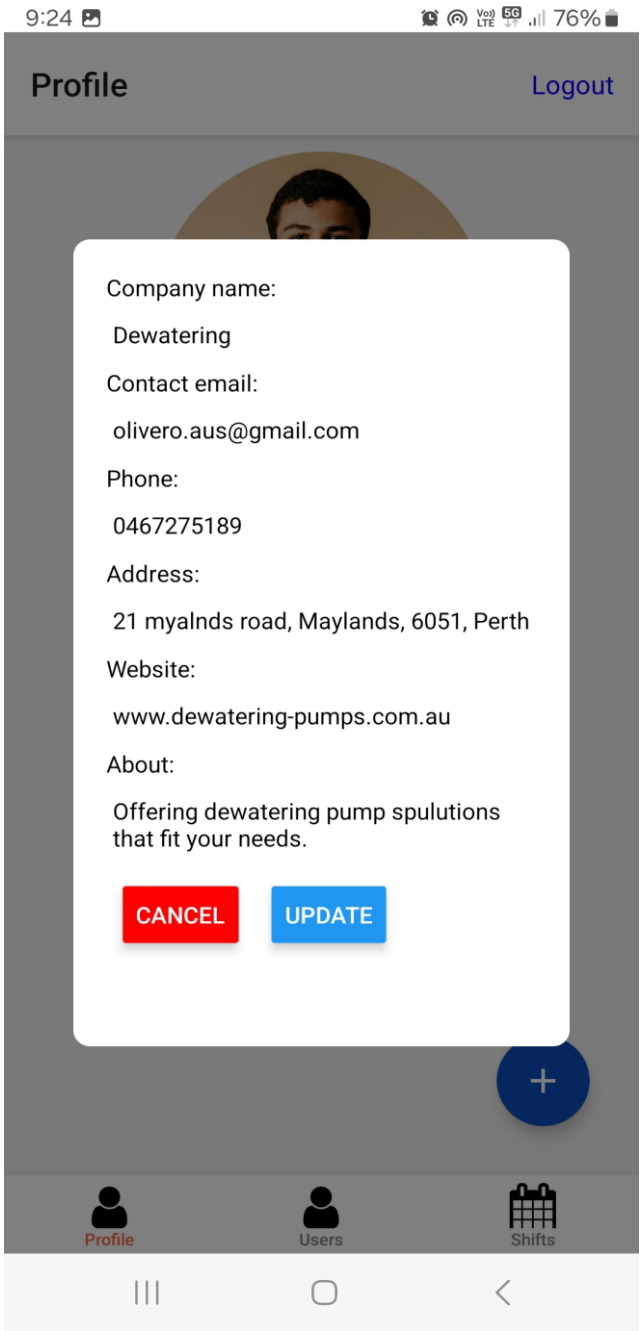


Figure 4. Login Page

Figure 5. Add Company

The company user has many functionalities: when first logging in it has to create a company profile(minimum company name and contact email) so that the company has an ID that will link any user created to this company. Now that a profile has been created this user can: update company profile, create sub company users, create users, assign to users a shift(not to sub company users) based on their availability, modify/delete any shift by looking at the calendar view and also approve of any shift(post approval might also be implemented in the future). The screen shot highlight company profile creation, user creation and shift allocation.



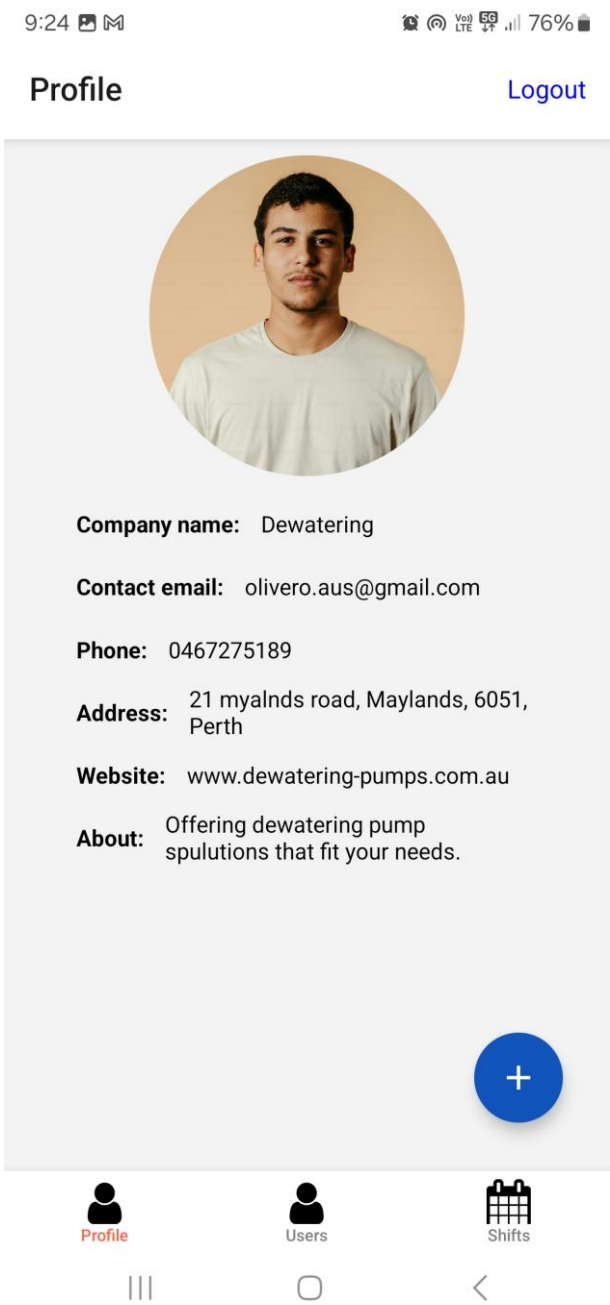
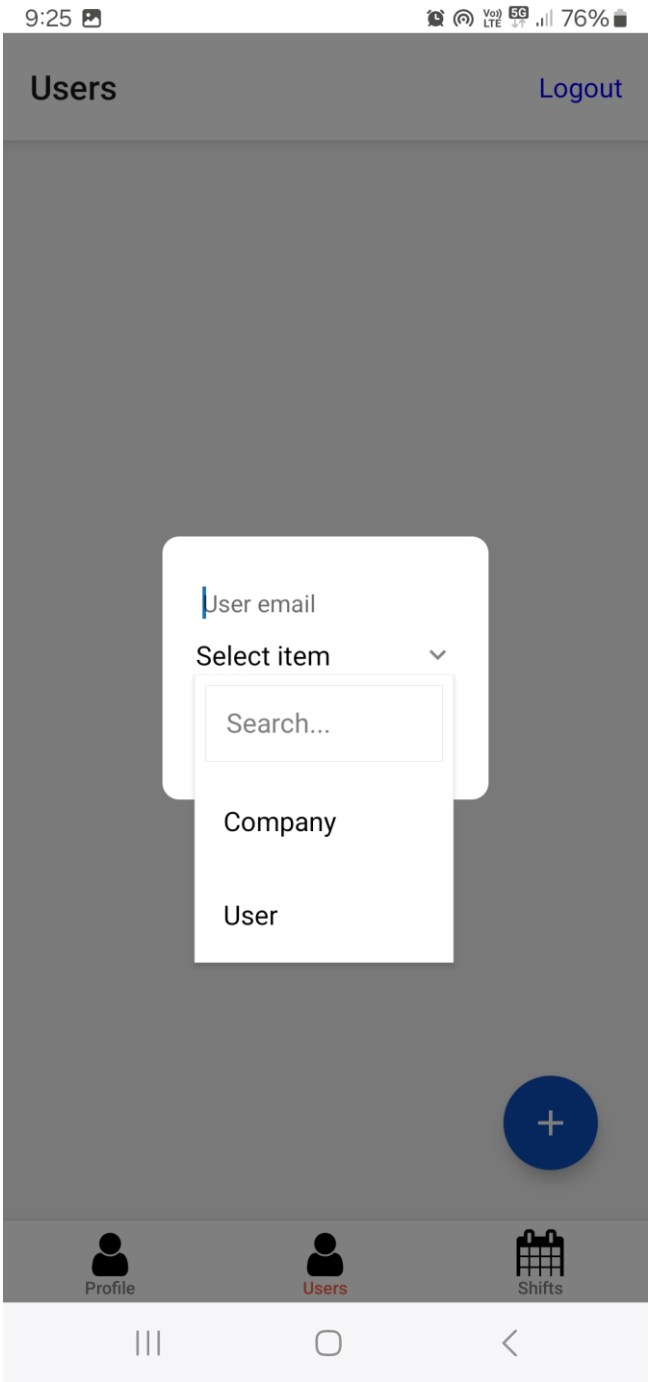


Figure 6. Company Profile Detailed Information

Figure 7. Company Profile



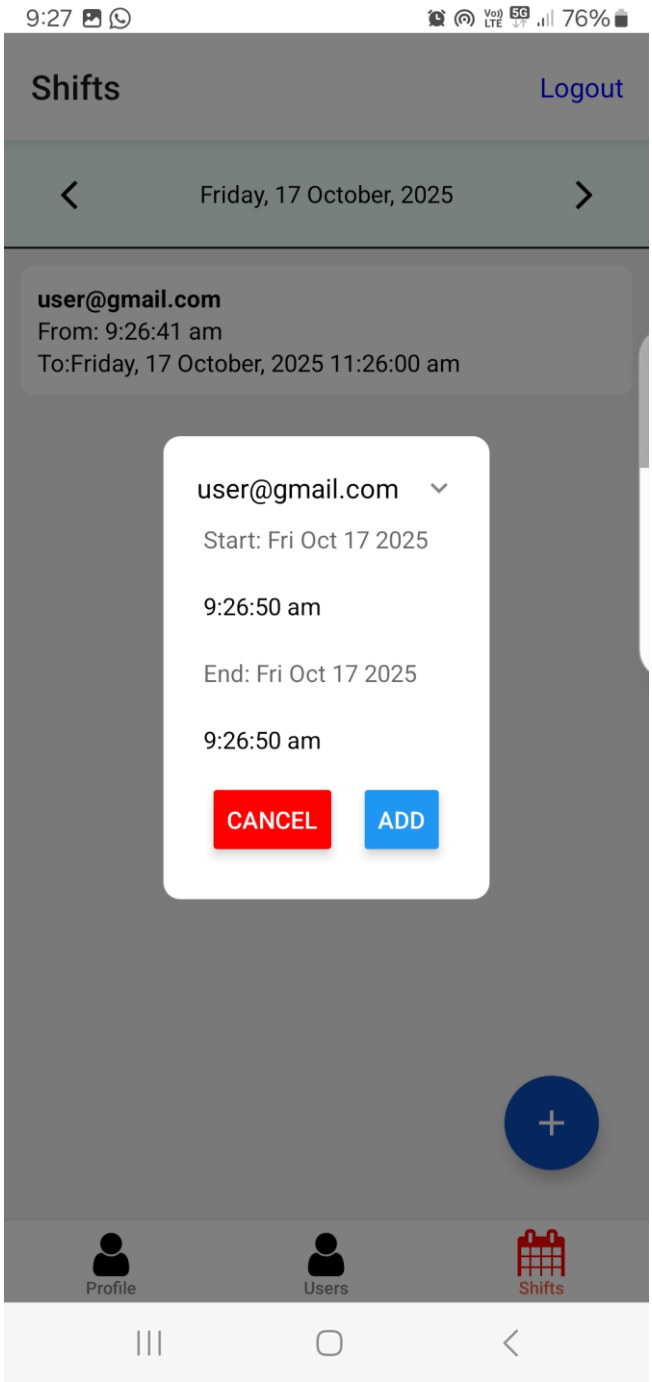


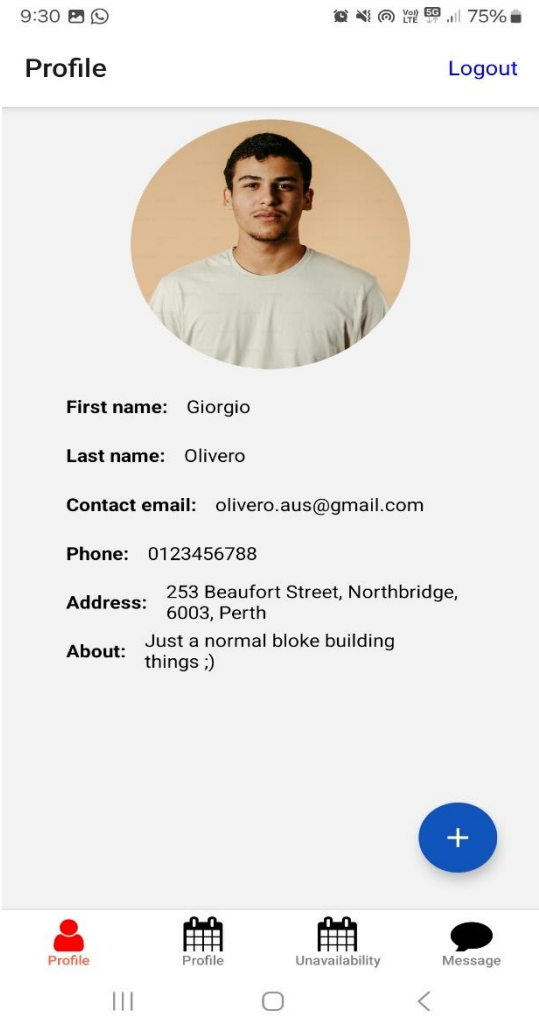
Figure 7. Add New User

Figure 8. Add Shift for User

The sub-company user can only see the company profile but cannot make any updates, is able to create shifts that will be added to the database, but needs approval from the main company user. A message board will also be implemented in future releases.

Finally, the company user will be able to see his profile and update it(first name, last name and email required), see the shifts he is rostered for and also post and comment in the message board(for example, to ask if someone can cover one or multiple of his shifts).

All users are able to reset their passwords through the link on the login page. Please check the spam folder, as the email for password reset might be flagged as spam.



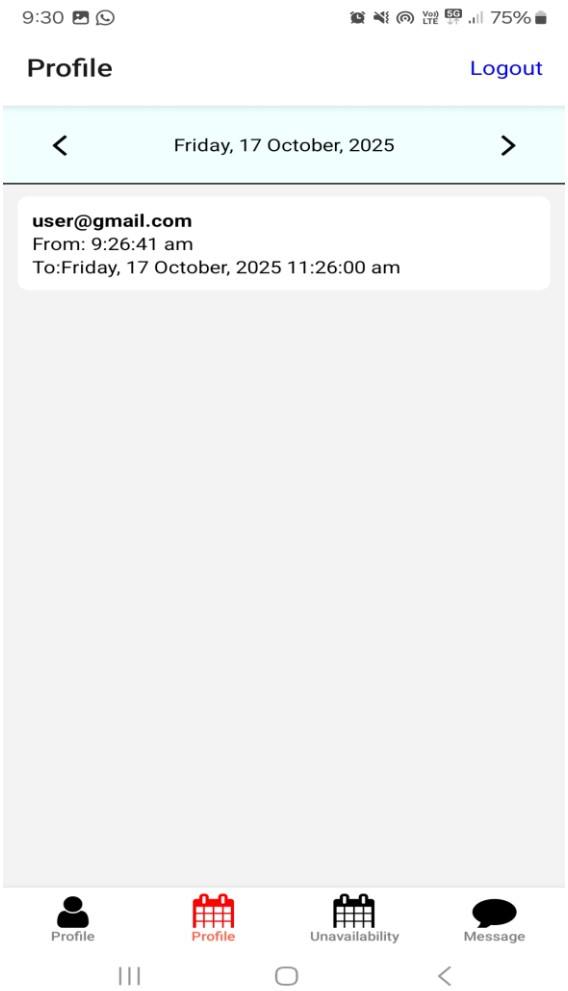
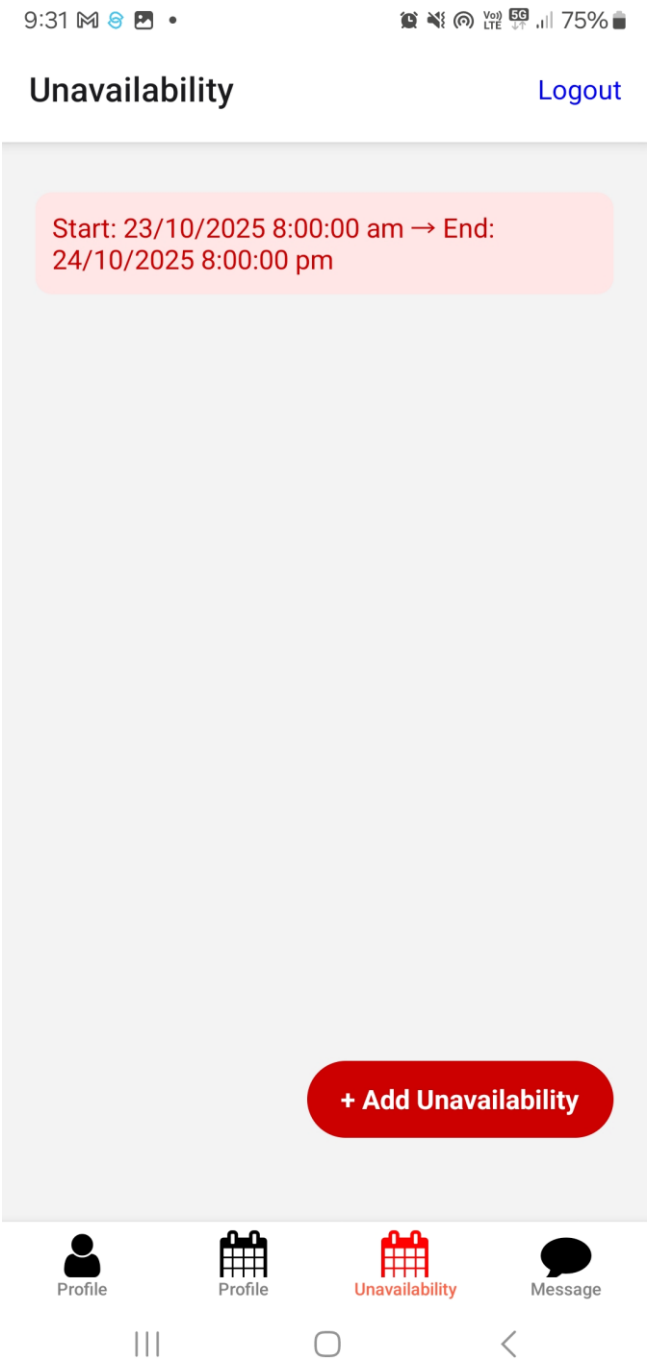


Figure 9. User Profile

Figure 10. Shift View for Users



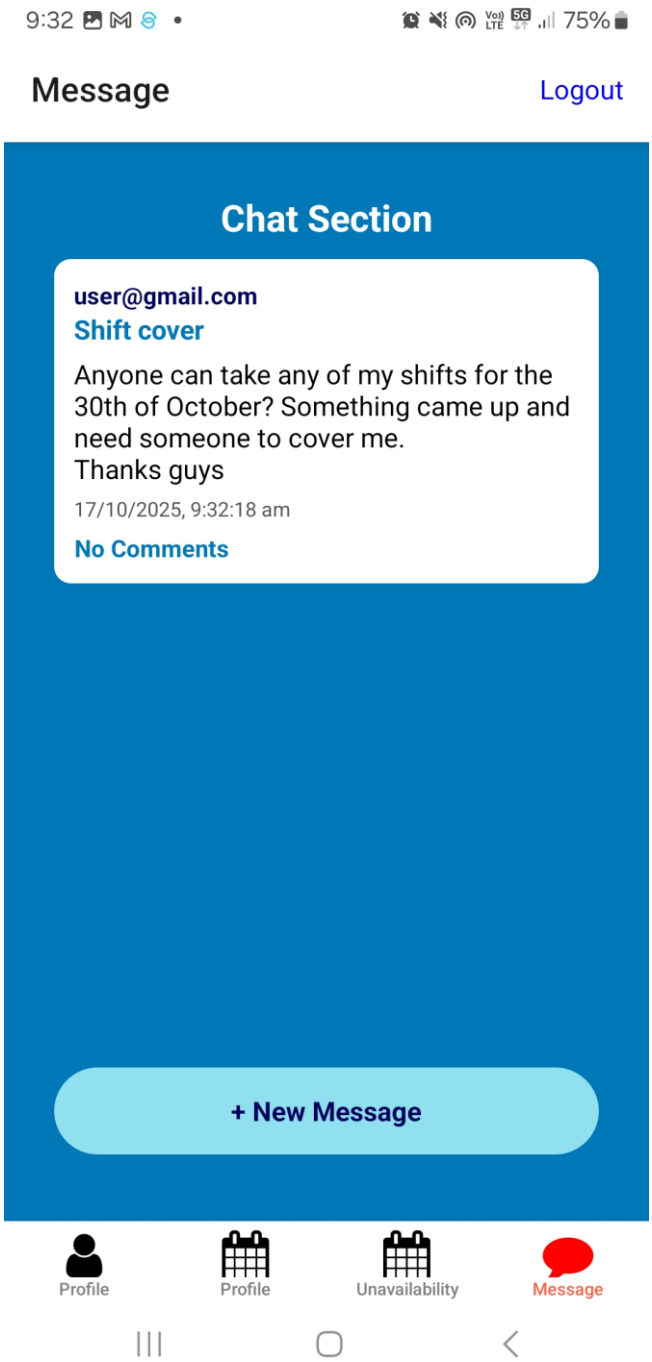


Figure 11. User Unavailability

Figure 12. Company Message Board

6. Discussion

6.1 Ethics

RoosterFlow is made to simplify the life of the general public and take into consideration of their perception on our software.

ACS Code of Ethics

We, as a group, have to be aware of the data the user is sharing with us and be mindful of confidentiality and data privacy regulations, and handle the data to comply with current standards(Australian Privacy Act), specifically when handling Personal Identifiable Information (PII). This also means that public information should be well protected, and a security system should be in place to protect against illicit acquisition of data.

Curtin Integrity

Plagiarism: In our group, we ensure that the source we use for our project is documented and cited accordingly.

Use Of AI: We ensure that there is no use of AI being used in our project specifically in the coding part, that there is no use of genAI. If AI is used, the members would have to disclose it to the group, and changes to the content would be made accordingly.

Concerns

Opinions and suggestions made by the stakeholders would be taken into account during sprint reviews meeting. Australian Privacy act would be enforced in the project regarding PII. Members in the group are individually responsible for ensuring they don't commit plagiarism or use AI, if done so, they have to disclse the misconduct for ethics and transparency.

Privacy & Security Concerns

PII must be primary concern, not just securing unwanted access but also storing, deletion and access from customers. API should not be publicly accessible, if possible, it should be restricted immediately. Identification system must be used for auditing rather than a Shared IDs, which would simplify the process.

A monitoring system should be in place to monitor the access to PII to simplify auditing in case of a breach (ISO 27001). To identify what data needs to be protected, a register of personal data collection can be kept. To limit the amount of personal information needed PII should be encrypted with restricted access based on user role. The PII would be deleted when the user is deleted from the system (Australian Privacy Principle). API can be accessed only with an authorization token with restriction access based on role. For the companies, sub-companies are created to better identify who is the company is accessing the data.

6.2 Sustainability

With the introduction of RoosterFlow App, there would dbe various impacts on economic, social and Environmental. The app also ensure to involve indigenous people throughout the data collection process to ensure their priorities are met. The data collected would be with their consent and it would be transparent on how it is used and accessed.

Economic Impact

The app would allow reduced admin time thus lowering cost and increasing productivity. It would also reduce the reliance on hard copies thus reducing the cost significantly.

Social Impact

The app is readily accessible across all devices. With its easy to navigate and guide, it leads to better rostering which would reduce the misunderstanding that often happens in workplace and in-turn lead to happier staffs. With the app's communication system in-place, team communication is easy.

Environmental Impact

With the digital footprint, the app reduces the paper wastage. With the unavailability feature in the app, it reduces carbon footprint. Rather than the employee let the higher management know of their unavailability, by either calling or sending a mail, with this feature, the employee would only have to set their unavailability and if the manager tries to put a shift in the timeslot the employee is unavailable, it won't be possible.

6.3 Data Security

The project applied several good practices for data protection:

- All communication between client and server happens over HTTPS.
- Authentication and Authorisation for protecting APIs by JWT Token. Spring Security was used to protect user data and control access to the system using a JWT Token.
- Passwords are never stored in plain text(Encrypted).BCrypt encryption was used to store passwords securely.
- Role management and access management(Admin,Company,Normal User).

- Only authorised users can access or reset passwords through secure email links.

6.4 Project Management

At the beginning of the semester, the project proposal included several features:

- Building a user management and authentication system.Authentication process for all users: register, login, logout, forgot password and reset password via email
- Creating an admin panel: add company, edit company, deactivate company
- Creating company panel: add employee, edit employee, deactivate employee, add/remove shift and edit shift
- Creating employee panel: set unavailability, view shifts, view profile
- Check-in, Check-out by location and face recognition
- Advanced Reports
- Deploying the application online

7. Conclusion

Compared to the original timeline, we were definitely behind schedule. We implemented all the basic front-end functionalities for all the users(also added a user in the middle of the project), but we were a few weeks late. The back-end was implemented and integrated successfully, with a deployment phase also conducted online, not just locally. The database needs to be moved to PostgreSQL for production, and the front-end application needs to be deployed on the Play Store and Apple Store. A privacy consent must be added to the login page, and user information must be encrypted at rest on the backend database.

As for additional functionalities that were planned, we only managed to implement the message board, which is now visible and usable only on the user page (not implemented on the company side). Testing has been implemented only for some of the backend APIs, but needs to be done on a larger scale to test system-wide functionalities, not just back-end, but also front-end, especially when there is user validation, as that could pose a security issue (e.g., SQL injection, cross-site scripting).

As of today, a working prototype exists, which was the goal from the beginning. However, as mentioned earlier, the division of labour has not been equal from the start for various reasons. Experience played a factor, but some people also took considerably longer to accomplish tasks.